Project 2 – Acme Analytics Linux Platform

Day 3 · Phase 1A: the service desk, building svc01, growing the disk, the network story

Muhammad Bilal Javed · 16–17 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1A – Service desk: GLPI tickets, knowledge base and asset inventory
Host worked on	svc01 (new VM)
Status	Step 1A.1 🟡 – svc01 installed, networked, names and SSH key working, disk grown to 23 GB, fully patched; NAT-adapter removal and snapshot next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary.

---

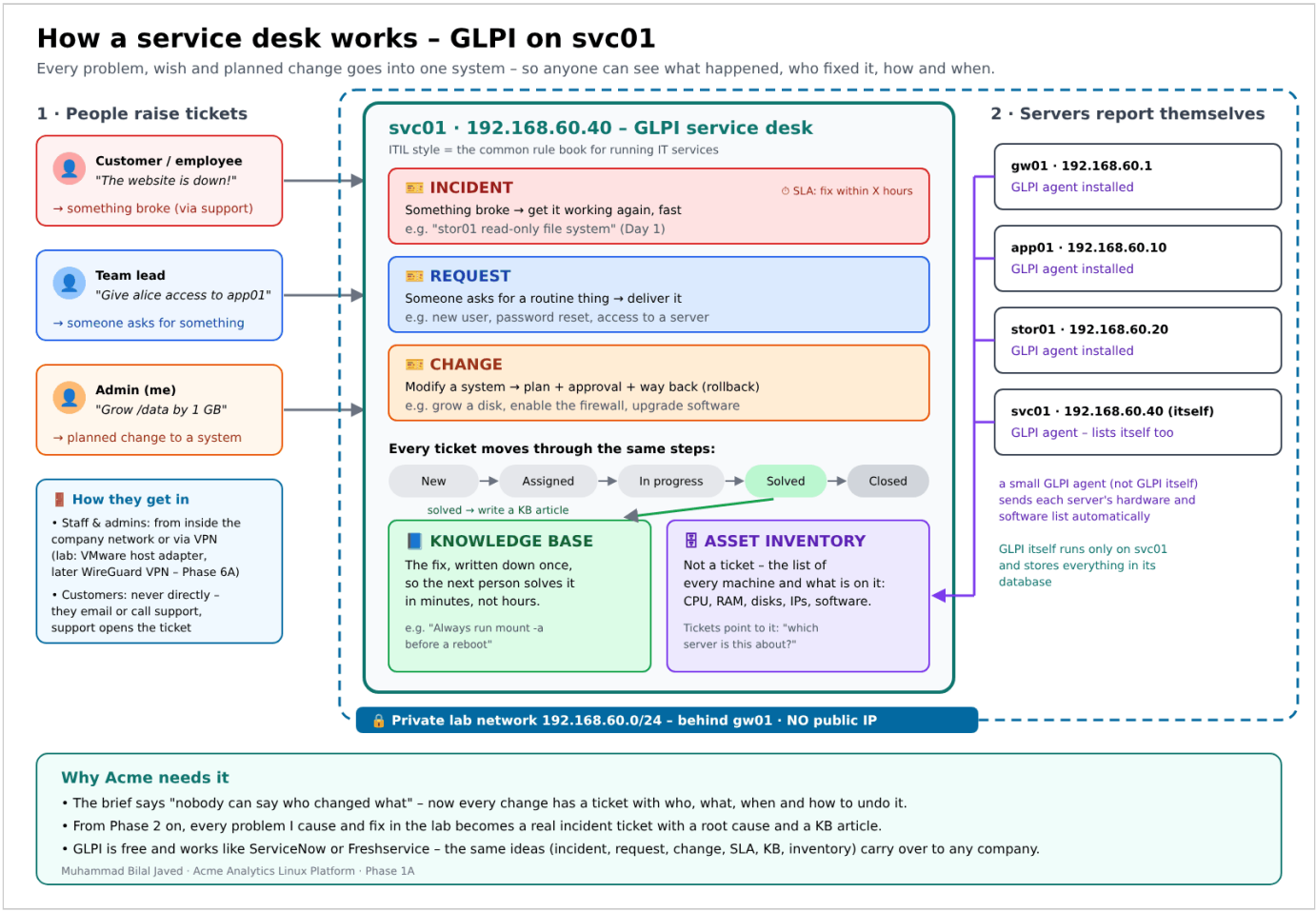
1. The story – why a service desk, and why now

💡 **In plain words:** until now I fixed problems and wrote them down in these reports. A real IT team does not work like that: every problem, every change and every "please give me access" goes into a **ticket system**, so anyone can see *what happened, who fixed it, how, and when*. Phase 1A gives Acme that system.

Where it fits in the Acme story. The brief says: *"nobody can say who changed what"* and *"the first enterprise customer's security questionnaire asks questions the team cannot answer."* The network (Phase 1) now works, so this is the right moment: from Phase 2 on, **every incident I cause and fix becomes a real ticket**, every planned change (e.g. "grow /data by 1 GB") becomes a **change** ticket with a rollback plan, and every server appears in an **asset inventory** automatically.

Ticket type	Meaning	Example in this project
Incident	Something broke	"stor01 read-only file system" (happened on Day 1)
Change	A planned change with a plan and a rollback	"Grow /data by 1 GB", "Enable UFW on stor01"
Request	Someone asks for a routine thing	"Give alice access to app01", a password reset
Knowledge base (KB)	The written fix, so the next person solves it faster	"Always run `mount -a` before a reboot"
Asset inventory	Every server with CPU, RAM, disks, IPs and installed software	gw01, app01, stor01, svc01

📊 **The picture – how a service desk works:**



How a service desk works – GLPI on svc01

Key understanding (in my own words).

> In the IT industry a problem is an **incident**, and handling them is **incident management**. When someone asks for a routine thing – a new user, a password reset, access to a server – it is a **request**. When I modify a system itself – grow a disk, enable a firewall, upgrade software – it is a **change**, which needs a plan, approval and a way back. All three are **tickets** in GLPI, and every ticket moves through the same steps: new → assigned → in progress → solved → closed; a solved fix is written down in the **knowledge base**. The **asset inventory** is not a ticket: it is the list of every machine and what is on it, filled automatically by the GLPI agent on each server, so a ticket can say which server it is about. GLPI follows **ITIL**, the common rule book for running IT services, and is used by many European companies and public bodies.

Who can reach svc01? svc01 has **no public IP** – it sits on the private lab network behind gw01, like app01. Staff and admins reach it from inside the company network (in the lab: my PC through VMware's host adapter; from Phase 6A through the WireGuard VPN). **Customers never reach it directly**: they email or call support, and support opens the ticket for them. A service desk holds the details of every server, so exposing it to the internet would be a security risk.

Why svc01 appears twice in the picture: in the middle it is the **GLPI application** (the help desk); on the right it is the **machine** svc01, which also runs a GLPI agent and lists itself in the inventory. GLPI itself runs only on svc01; the other servers get only the small agent.

Incident or change? "The server is broken, please update it" is an **incident** (something is broken). "The server runs an old version, upgrade it next Tuesday" is a **change** (planned). The SLA – how fast a ticket must be handled – is the IT team's promise, not something the customer sets.

Why GLPI. GLPI is a free, ITIL-style service desk used by many European companies and public bodies. Freshservice or ServiceNow need a paid licence; GLPI teaches the same concepts – incident, change, request, SLA, KB, CMDB – and they carry over to any tool.

Why it matters for my job search. Many of the junior IT / DevOps postings I reviewed ask for ticket systems, SLAs, asset management or knowledge articles. In an interview I can say: *"I ran my lab through GLPI. Every incident I caused has a ticket, a root cause and a KB article, and every server is in the inventory."*

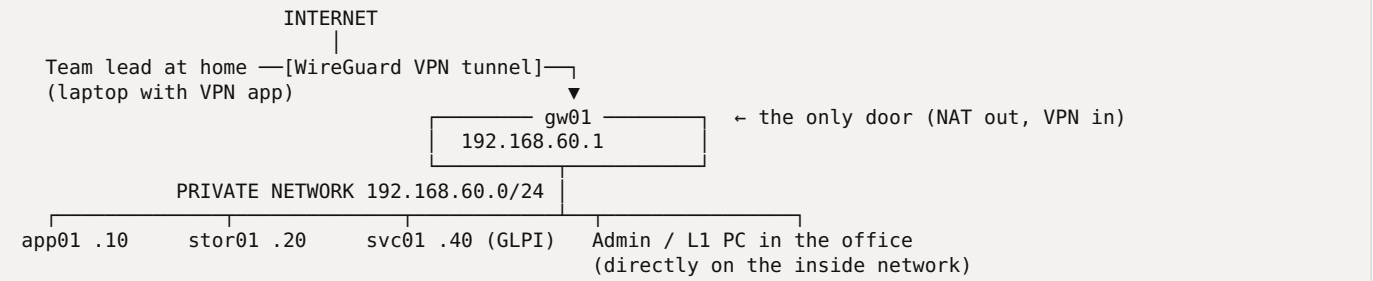
Why a separate server. The service desk must keep working when the servers it tracks are broken – if app01 is down, I still need to open the incident ticket. So it gets its own VM, `svc01`. Later, Project 1 reuses the same svc01 as a second "customer" – one service desk serving two customers is how IT system houses and managed service providers work.

2. svc01 in the lab

VM	Role (plain words)	Role (technical)	IP
gw01	The gatekeeper	NAT, firewall	192.168.60.1
app01	The shop window	App server	192.168.60.10
stor01	The archive room	Storage, backups, logs	192.168.60.20
svc01	**The help desk**	**GLPI tickets, KB, inventory (Docker)**	**192.168.60.40**

2a. How people reach svc01 – and where gw01 comes in

💡 **In plain words:** gw01 is the **only door** between the private network and the internet. People already **inside** (in the office) go straight to svc01. People **outside** (at home) must come in through the door with a **VPN** – like a company badge for remote workers. Strangers and customers never get in.



Who	Where	Path to svc01	Through gw01?
Employee / team lead in the office	On the company network	Browser → `http://192.168.60.40`	❌ already inside
Team lead / admin from home	On the internet	VPN app → encrypted tunnel to gw01 → private network → svc01	✅ via the VPN
Stranger on the internet	Outside, no VPN	Blocked – svc01 has no public IP, gw01 lets in VPN users only	🚫
Customer	Outside	Never connects – emails or calls support; L1 opens the ticket	—
Me in the lab, now	Windows PC	VMware host adapter on VMnet2 → browser → `http://192.168.60.40`	❌
Me in the lab, from Phase 6A	Like the home worker	WireGuard VPN to gw01 → svc01	✅

Lab note: VMware may give the Windows host adapter on VMnet2 the address 192.168.60.1 – gw01's address. Before opening GLPI, check it in the Virtual Network Editor / Windows network settings and use a free address such as 192.168.60.250.

2b. Three examples, start to finish

How the GLPI agent fits in: a small program on each server looks at its own machine once a day (CPU, RAM, disks, IP, software) and sends the report to `http://svc01` – finding svc01 through `/etc/hosts` or DNS. GLPI stores it as an **asset**. The agent creates no tickets and nobody logs in through it. Tickets come from **people**; in each ticket the person picks the affected server ("Item: app01") from the inventory, which links the ticket to the machine.

Thing	Question it answers	Used by
`/etc/hosts` / DNS	"What is stor01's IP?"	Machines, to connect
Asset inventory	"What is app01, what does it have?"	People, to know and plan
Ticket	"What is wrong or needed, and who did what?"	People, to work

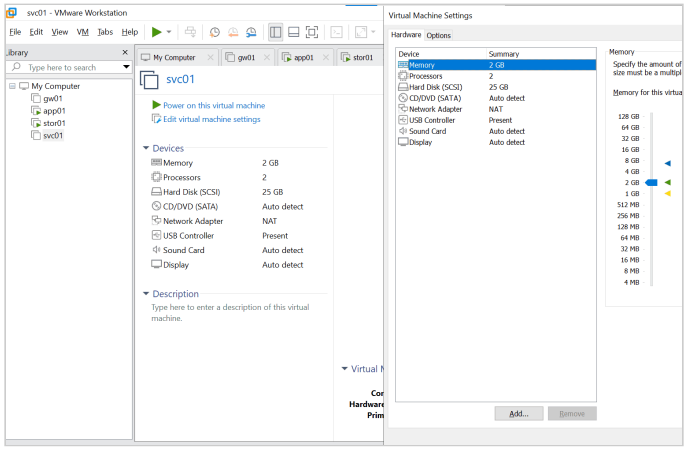
1. **Alice (employee) → Incident.** "I can't open the shared folder on app01." Item: app01. L1 checks (app01 pings, folder hangs) → escalates to **Linux Admins (L2)** with notes → L2 finds the firewall blocking NFS on stor01, fixes it, writes the root cause → Solved → KB article → alice confirms → Closed.
2. **Team lead → Request.** "New developer carol starts tomorrow; account on app01 in group devs." L1 runs `useradd -m -s /bin/bash -G devs carol` and `passwd carol` → Solved → Closed. When carol leaves: a new request → `usermod -L carol`.
3. **Admin → Change.** "/data on stor01 is 90 % full." The inventory shows 4 GB free in the storage pool → change ticket "Grow /data by 1 GB" with plan (`lvextend -r -L +1G`), risk, rollback (LVM snapshot first) and approval → done → `df -h` checked → Closed. The next agent run shows the new size automatically.

In the lab I play every role with separate GLPI accounts: `alice`/`bob` as requesters, `tech.bilal` in the groups **Service Desk (L1)** and **Linux Admins (L2)**. Escalation = reassigning the ticket to the higher group with notes; the history stays with the ticket.

3. VM settings

Setting	Value	Why
OS	Ubuntu Server **22.04.5** LTS (planned 24.04; the 22.04 ISO was used)	Same version as the other servers; GLPI runs in Docker either way
RAM	2 GB	GLPI + its database in Docker
CPU	2 vCPU	Smooth web interface; 1 would also work
Disk	25 GB	Database, ticket attachments, Docker images
Network	**One adapter, Custom → VMnet2**	Lab network only, like app01 and stor01 – internet only through gw01's NAT
CD/DVD	Ubuntu Server ISO, connect at power on	Installation medium
Sound card, USB	Optional – can be removed	A server does not need them
Hostname / user	`svc01` / `bilal_004`	Same naming pattern as the other servers
IP (set in the installer)	`192.168.60.40/24`, gateway `192.168.60.1`, DNS `8.8.8.8`	VMnet2 has no DHCP, so the address is set by hand
Installer option	✅ Install OpenSSH server	So gw01 can manage it over SSH with keys

📸 Evidence – VMware's default settings: 2 GB, 2 CPUs, 25 GB, but the network adapter is on NAT:

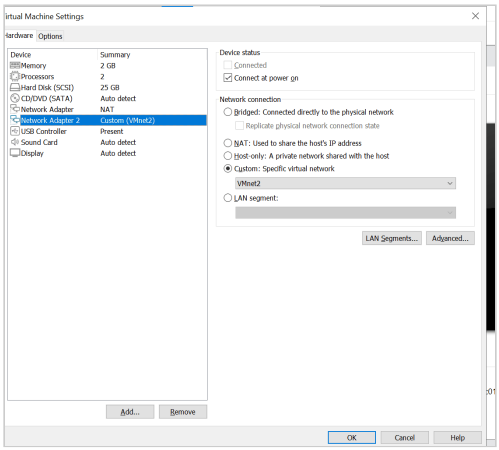


svc01 default settings with NAT

Issue: network adapter on NAT

Seen	VMware creates a new VM with its adapter on **NAT** (direct internet). Then a second adapter was added on VMnet2, so the VM had two adapters.
Why it matters	With NAT, svc01 would bypass gw01 – the gatekeeper design from Phase 1 would be broken, and the NAT adapter would have to be unplugged and netplan fixed after the install (the same trouble app01 had on Day 1).
Fix	Keep one adapter on **Custom → VMnet2**; **remove** the NAT adapter.
Lesson	Check every new VM's network before the first boot – the default is not the design.

📸 Evidence – Network Adapter 2 on Custom (VMnet2), connect at power on:

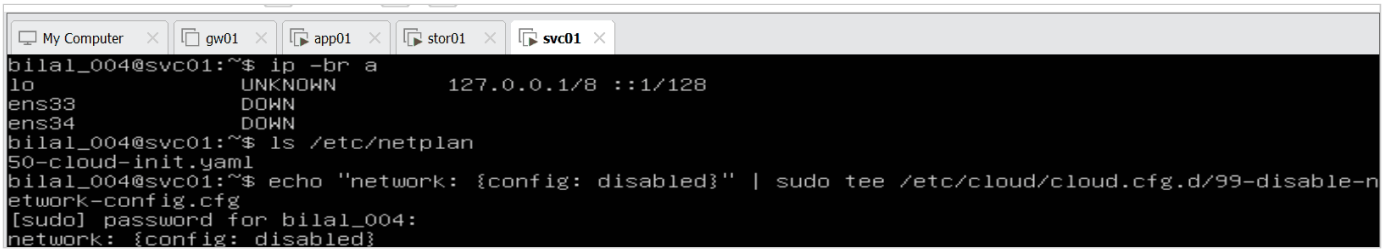


adapter on VMnet2

4. Installation and first boot

4.1 No network after the install

📸 **Evidence – two cards, both DOWN, no IP address; only one netplan file:**

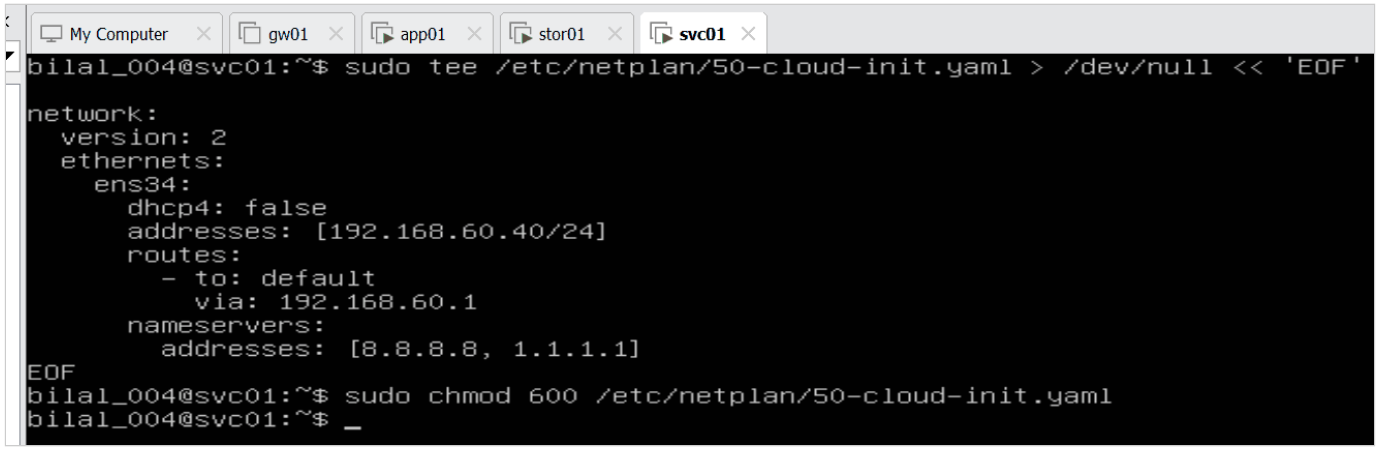


ens33 and ens34 down

Seen	`ip -br a` shows ens33 and ens34, both **DOWN** and without an address; `ip route` is empty; `ping` says *Network is unreachable*.
Cause	The NAT adapter was still attached during the install (two cards), and the manual network settings from the installer were not applied.
Fix	Stop cloud-init from managing the network (as on app01 and stor01, Day 1) and write the netplan file for ens34 (the VMnet2 card) by hand.

```
echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
sudo tee /etc/netplan/50-cloud-init.yaml > /dev/null << 'EOF'
network:
  version: 2
  ethernets:
    ens34:
      dhcp4: false
      addresses: [192.168.60.40/24]
      routes:
        - to: default
          via: 192.168.60.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
EOF
sudo chmod 600 /etc/netplan/50-cloud-init.yaml   # only root may read it
sudo netplan try                                 # applies it; reverts after 120 s unless I press Enter
```

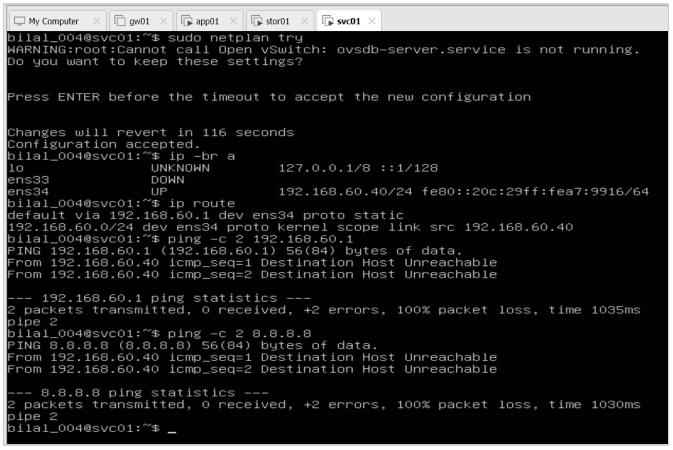
📸 **Evidence – the netplan file:**



netplan file for ens34

4.2 Address works – but gw01 was off

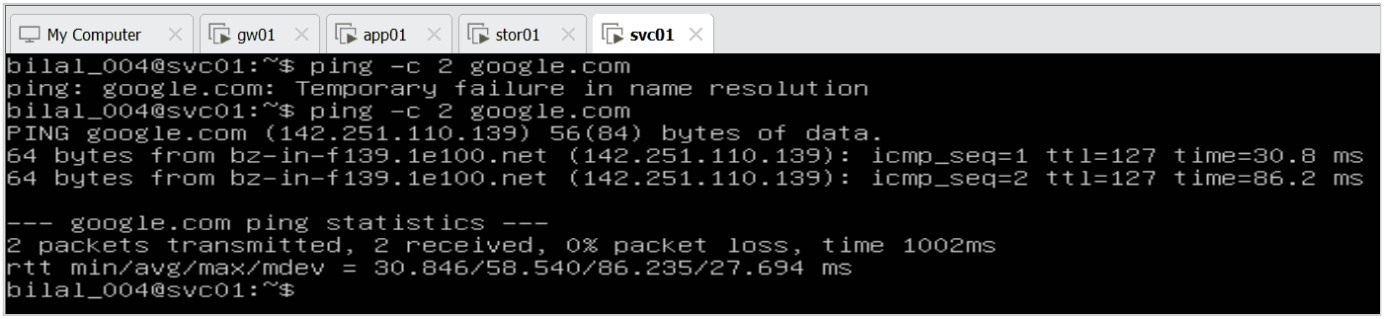
📸 **Evidence – `ens34 UP 192.168.60.40/24`, default route via .1, but "Destination Host Unreachable":**



netplan try, ping unreachable

Seen	svc01 has its address and route, yet `ping 192.168.60.1` and `ping 8.8.8.8` return *Destination Host Unreachable*, and `ping google.com` *Temporary failure in name resolution*.
Cause	**gw01 was powered off.** svc01 asked “who is 192.168.60.1?” and nobody answered. Without the gateway there is no way to the internet – and the DNS server 8.8.8.8 is on the internet.
Fix	Power on gw01.
Lesson	**Always start gw01 first.** The servers can talk to each other without it, but not to the internet or DNS.

📸 **Evidence – after starting gw01: google.com resolves and answers:**



DNS and internet through gw01

4.3 Names: `/etc/hosts` on all four servers

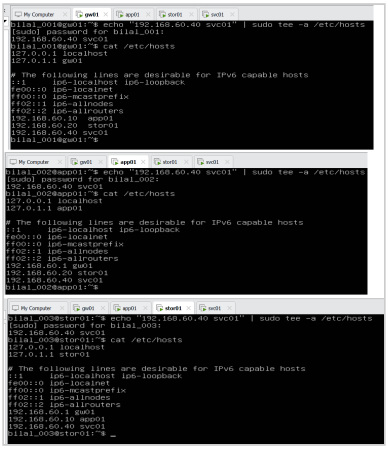
Each machine has its **own** contact list, so the new server must be added to the others, and svc01 must learn the others. A machine never lists itself – it already knows itself as `127.0.1.1`.

Machine	Lists
gw01	app01, stor01, **svc01**
app01	gw01, stor01, **svc01**
stor01	gw01, app01, **svc01**
svc01	gw01, app01, stor01

```
echo "192.168.60.40 svc01" | sudo tee -a /etc/hosts      # on gw01, app01, stor01
echo "192.168.60.1 gw01"    | sudo tee -a /etc/hosts     # on svc01 (one line per echo)
echo "192.168.60.10 app01"  | sudo tee -a /etc/hosts
echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts
```

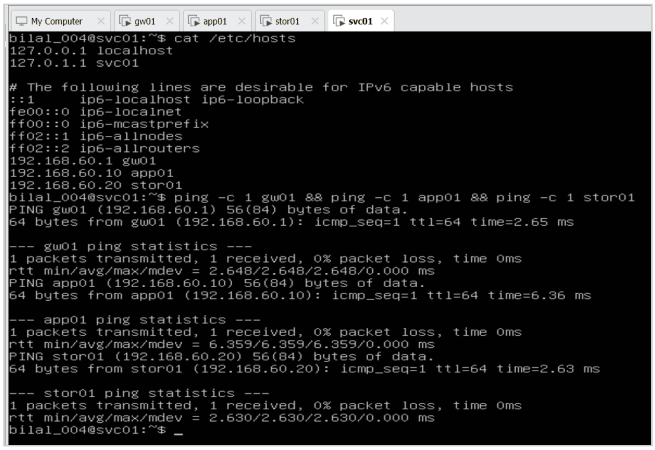
`printf "a\nb\n"` can write several lines in one command because it understands `\n` (new line); plain `echo` does not.

📸 **Evidence – svc01 added on gw01, app01 and stor01:**



hosts on the other three servers

📸 Evidence – svc01's list and ping by name to all three (`ttl=64` = same network, no router in between; `&&` = run the next command only if this one worked):

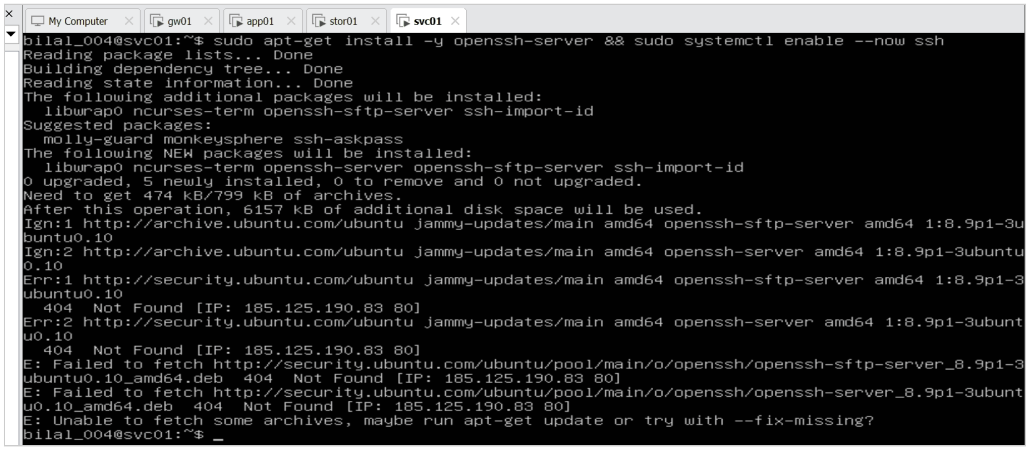


svc01 hosts and ping by name

4.4 SSH: package list out of date

Seen	`apt install openssh-server` fails with **404 Not Found**.
Cause	Not a network problem – the server was reached. The **package list was old**: it asked for a version that has since been replaced, so the file no longer exists.
Fix	`sudo apt update` first, then install.
Lesson	On a fresh server, always run `apt update` before `apt install`.

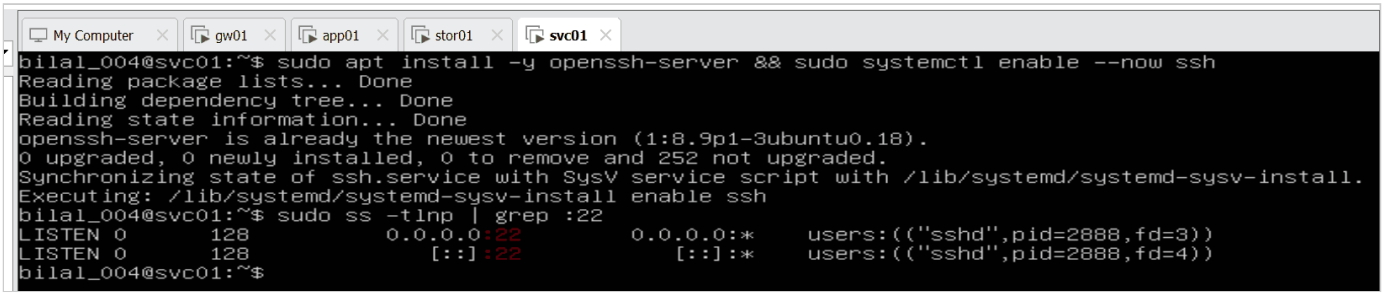
📸 Evidence – the 404 (note `jammy` = Ubuntu 22.04):



apt 404

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo ss -tlnp | grep :22
```

📸 Evidence – sshd listening on 0.0.0.0:22 and [::]:22:



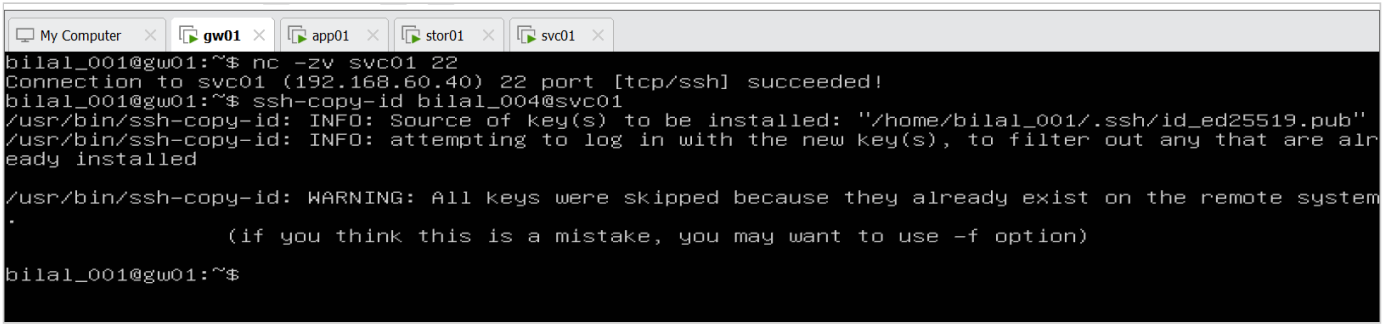
sshd listening

4.5 Key login and shortcut from gw01

```
nc -zv svc01 22                 # door open?
ssh-copy-id bilal_004@svc01     # copy bilal_001's public key
cat >> ~/.ssh/config << 'EOF'
Host svc01
    HostName 192.168.60.40
    User bilal_004
EOF
ssh svc01
```

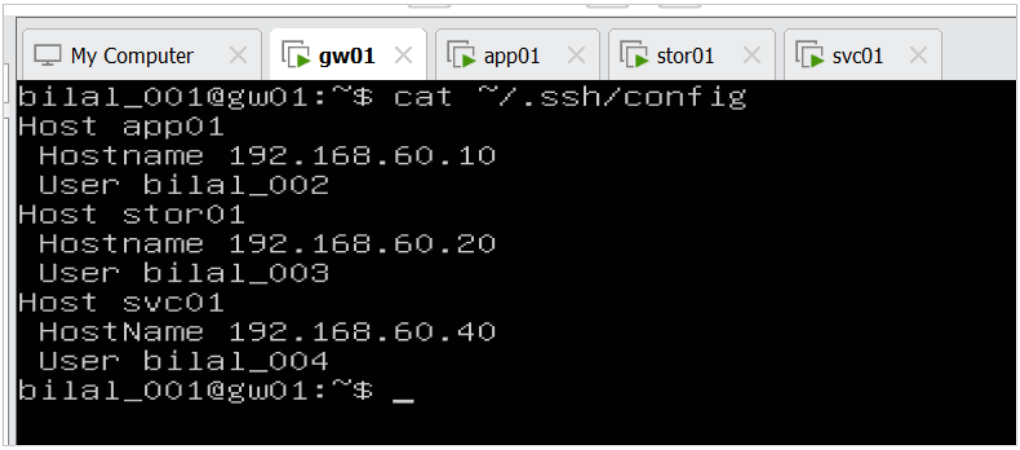
`ssh-copy-id` reported *"All keys were skipped because they already exist on the remote system"* – not an error: it first tests whether the key already works, and it did, so it avoided a duplicate.

📸 **Evidence – port 22 open, key already present:**



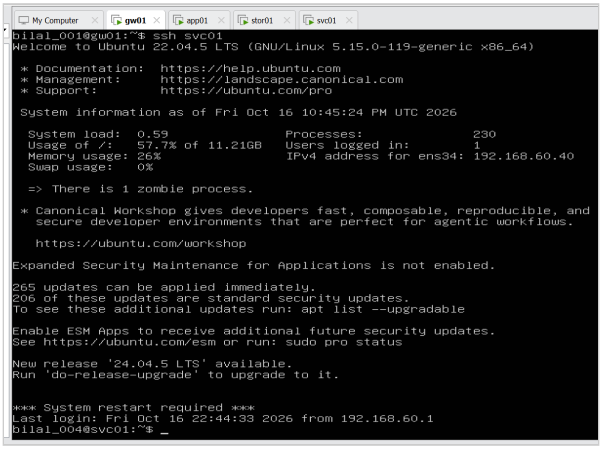
nc and ssh-copy-id

📸 **Evidence – gw01's SSH address book now has app01, stor01 and svc01 (keywords are not case-sensitive: `Hostname` = `HostName`):**



ssh config

📸 **Evidence – `ssh svc01` logs in as bilal_004 without a password:**



ssh svc01

4.6 Read from the welcome screen

Line	Meaning / action
Ubuntu 22.04.5 LTS	Same release as the other servers
Usage of /: 57.7 % of 11.21 GB	The installer used only about 11 GB of the 25 GB disk for the system; the rest is free in the LVM pool → **grow it before Docker** (`lvextend -r -l +100%FREE`)
265 updates, 206 security	Run `sudo apt upgrade -y`
New release 24.04 available	**Not** upgrading – all servers stay on 22.04
1 zombie process	Harmless; gone after reboot (processes are Phase 4)
Times in UTC	Optional: `timedatectl set-timezone Europe/Berlin`, so ticket times match local time
System restart required	Covered by the reboot after the upgrade

Key understanding (in my own words)

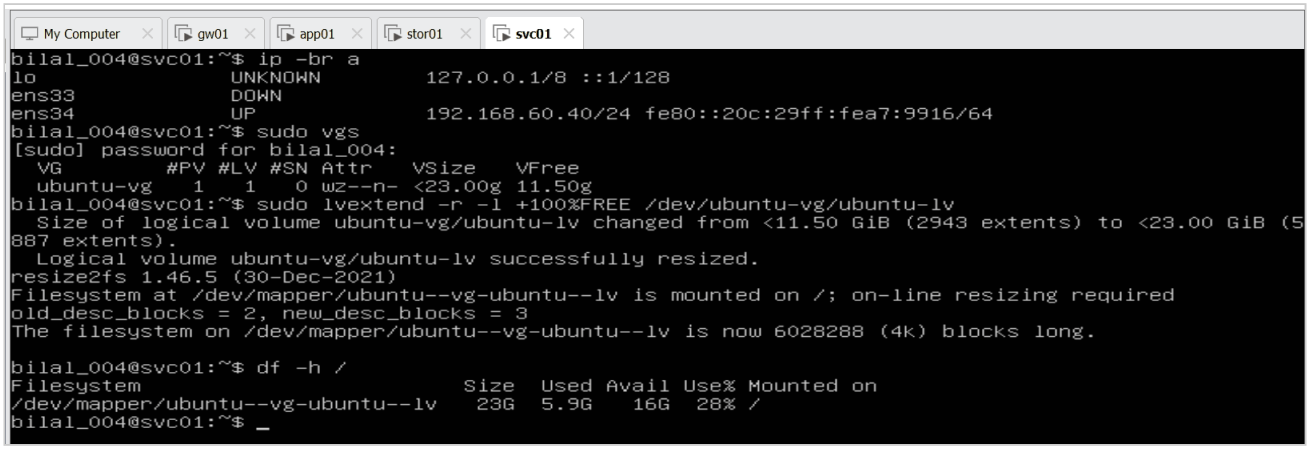
> svc01 joined the lab like app01 and stor01: one card on VMnet2, a fixed address set in netplan, and the default route pointing to gw01. Because gw01 is the only door, svc01 has no internet and no DNS while gw01 is off. Every machine has its own `/etc/hosts`, so a new server must be added to the others' lists and learn theirs. On a fresh server, `apt update` comes before `apt install`. With the key and the SSH shortcut, gw01 now manages all three servers by name, without passwords.

5. Growing the disk before Docker (LVM) – 2026-10-17

💡 **In plain words:** the welcome screen showed the system using only **11.5 GB of the 25 GB disk**. GLPI is about to move in – Docker images, a MySQL database with every ticket, user and inventory record, and ticket attachments – and all of it grows every day. A full disk makes MySQL crash, which would take the service desk itself down: exactly the Acme problem *"disks fill up without warning"*. So the disk was grown **before** installing anything. The system was also fully patched first, because GLPI will open a web service on port 80.

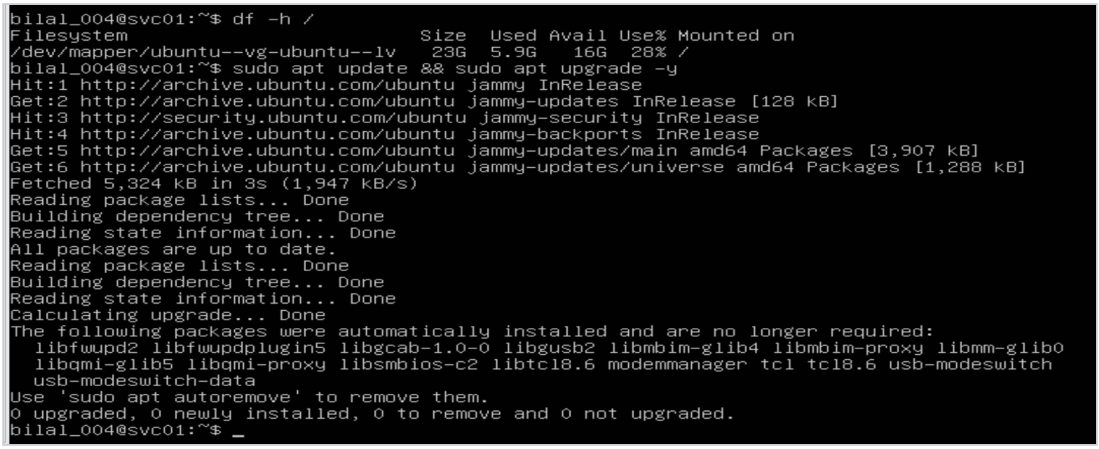
```
ip -br a                                                # ens34 192.168.60.40 (ens33 = old NAT adapter, to be
removed)
sudo vgs                                                # the pool: VSize 23 GB, VFree 11.5 GB unused
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv  # give the volume all free space; -r grows the file
system too
df -h /                                                 # 23 GB, 28 % used
sudo apt update && sudo apt upgrade -y                  # patch before opening a service
```

📸 **Evidence – `vgs` shows 11.5 GB free in the pool; `lvextend` grows the volume from 11.5 to 23 GB while running; `df` shows 23 GB:**



vgs, lvextend, df

📸 Evidence – all packages up to date:

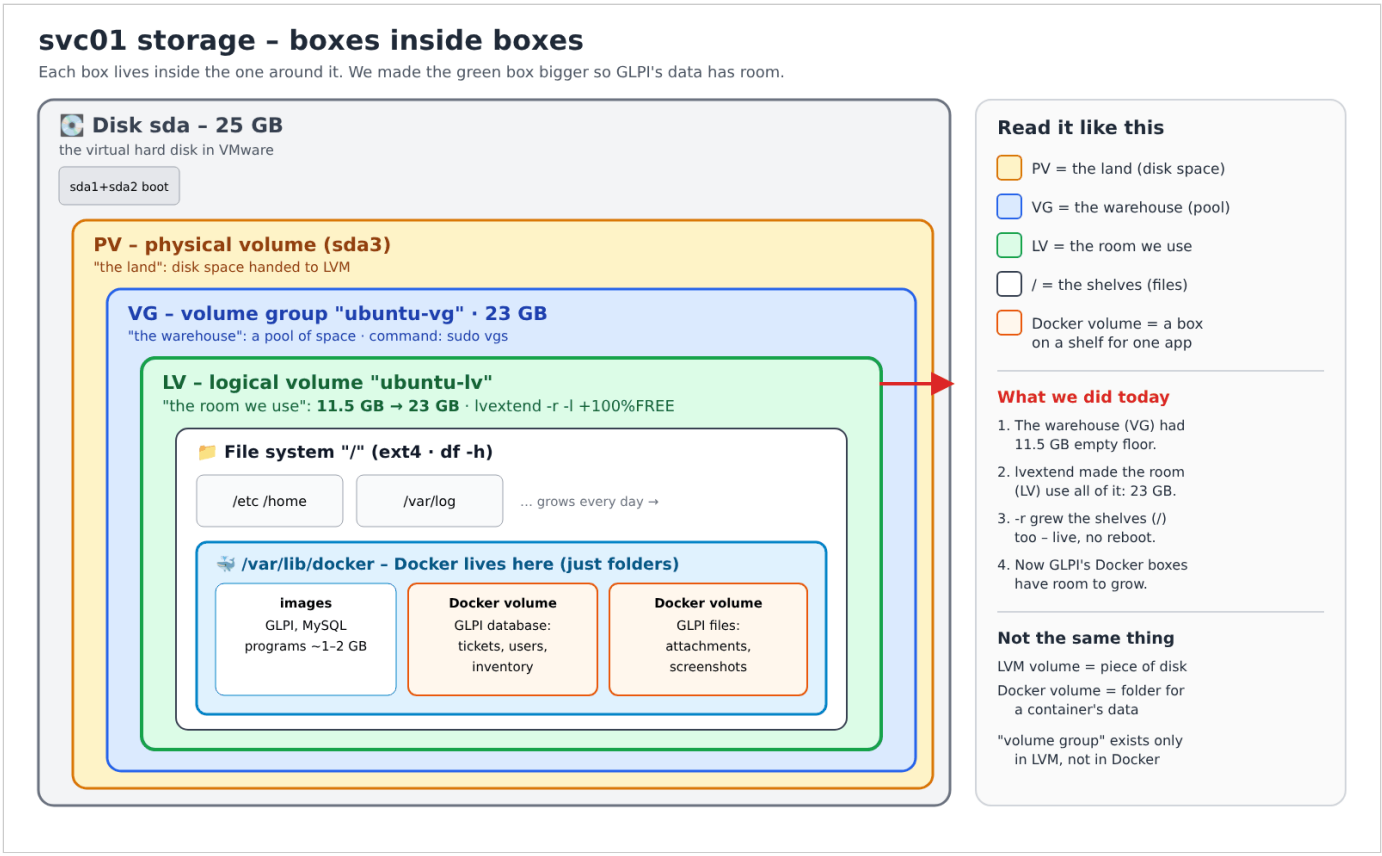


apt upgrade

Reading vgs

Column	Value	Meaning
VG	`ubuntu-vg`	Name of the pool
#PV	1	Built from 1 physical volume
#LV	1	1 logical volume cut from it
#SN	0	No snapshots
VSize / VFree	23 GB / 11.5 GB	Pool size / part not given to any volume yet

The picture – boxes inside boxes



svc01 storage layers

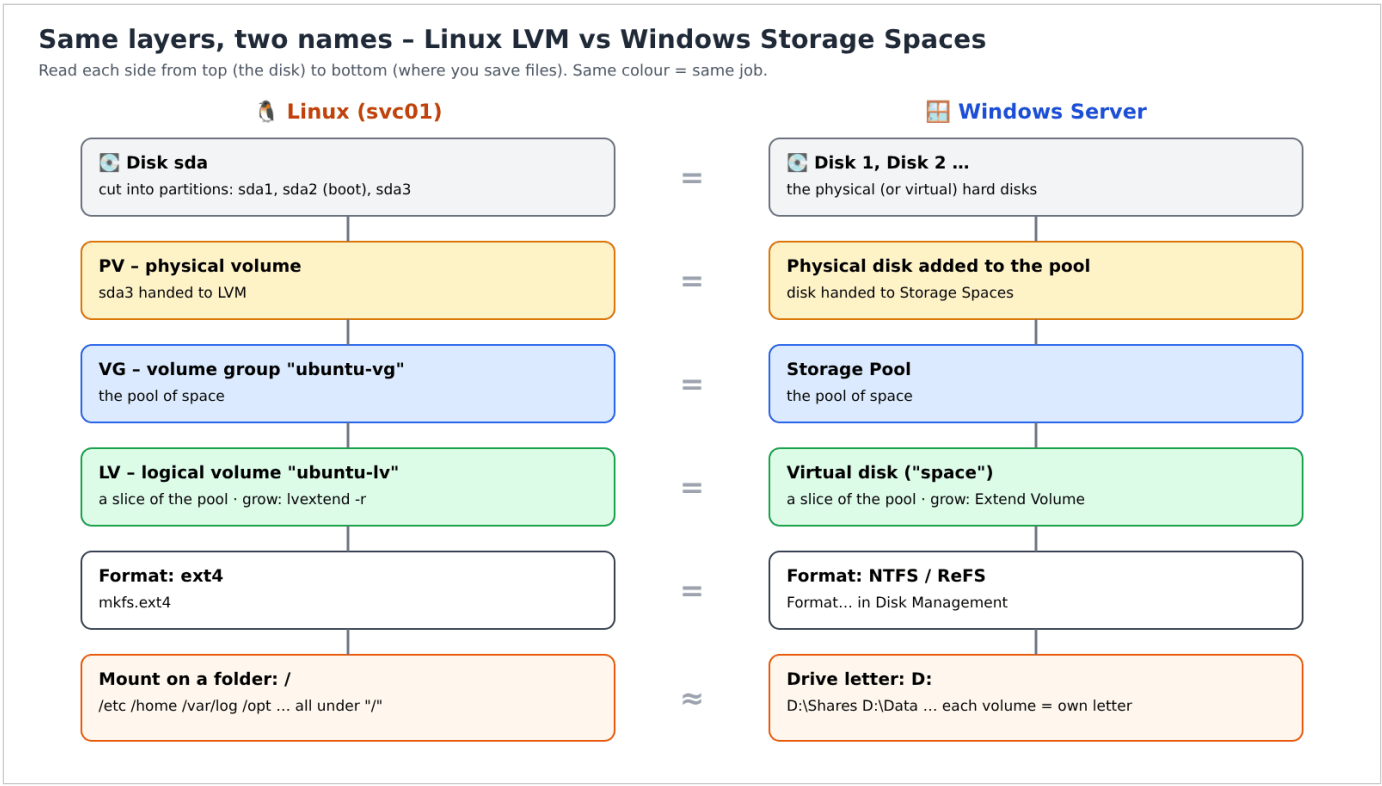
Part of the command	Meaning
`lvextend`	Make a logical volume bigger
`-l +100%FREE`	Small **l**: add all free space of the pool (capital `-L +5G` would add an exact size)
`-r`	Resize the **file system** too – otherwise the volume grows but the space is not usable
`/dev/ubuntu-vg/ubuntu-lv`	Path pattern `/dev/<VG>/<LV>`

Docker: image, container, mounted folder

Word	Simple meaning	Here
Image	The installer – a ready-made program package that never changes	`glpi/glpi`, `mysql:8.0` (kept in `/var/lib/docker`)
Container	The running program made from the image	GLPI and MySQL running
Mounted folder (volume)	A real svc01 folder shown inside the container, where it saves its data	`/opt/helpdesk/storage/mysql` → `/var/lib/mysql` inside MySQL

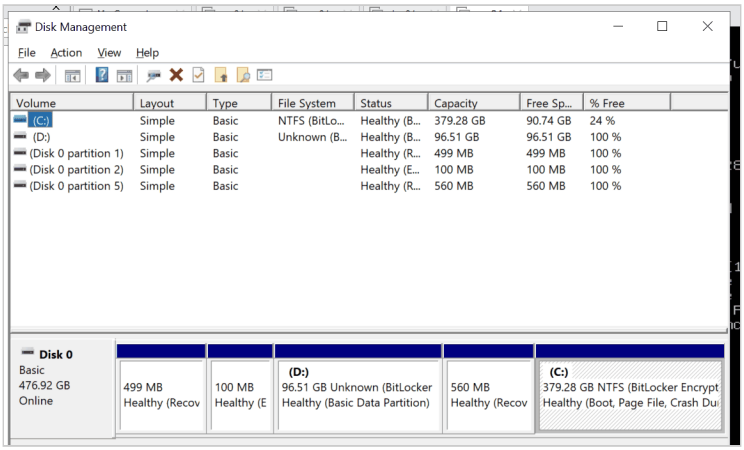
A container is thrown away when it is updated or recreated; data saved in the mounted folder on svc01 survives.

Same layers in Windows



Linux LVM vs Windows Storage Spaces

📸 On my own PC (Disk Management, `diskmgmt.msc`): a "Basic" disk – Disk 0 cut into partitions with drive letters (C:, D:), no pool layer:



Windows Disk Management

Where to look in Windows: **Disk Management** (Win + X) for disks, partitions and letters; **Storage Spaces** (Control Panel, or Server Manager → File and Storage Services → Storage Pools on Windows Server) for pools; PowerShell `Get-Disk`, `Get-Partition`, `Get-Volume`, `Get-StoragePool`.

Key understanding (in my own words)

> When I created svc01, I gave it one 25 GB virtual disk, **sda**. A disk can be divided into **partitions**, like C: and D: in Windows. Here, sda1 and sda2 are for booting, and **sda3** (23 GB) is used as a **physical volume (PV)**. On its own, a PV is just raw space. It is never divided: once it is added to a **volume group (VG)**, a pool of space, it belongs to that VG only. One VG can be built from one or several PVs. Here there is one VG, `ubuntu-vg` (23 GB), made from one PV. For a second VG, I would first create another partition on the disk, which becomes its own PV.
>
> Inside a VG I can create many **logical volumes (LVs)**. Here there was one, `ubuntu-lv`, with 11.5 GB, and the other 11.5 GB were free. I could have used the free space for more LVs; separate LVs give **isolation of space**: if one fills up (for example `/var/log` with logs), the others are not affected, and each LV can be grown, backed up or snapshotted on its own. This is about disk space, not security; data is protected by permissions, firewalls and encryption. On svc01 I chose to grow the single LV to the full 23 GB with `lvextend -r -l +100%FREE` while the system was running, and `-r` grew the file system too.
>
> An LV is only usable once it is **formatted** (ext4) and **mounted**; this one is mounted as `/`, the umbrella for every folder (`/etc`, `/home`, `/var/log`, `/var/lib/docker`, `/opt`).
>
> The same idea in Windows: physical disks (≈ PVs) are added to a **Storage Pool** (≈ VG) in Storage Spaces. The pool is cut into **virtual disks** (≈ LVs), which are formatted with NTFS or ReFS (≈ ext4) and given a

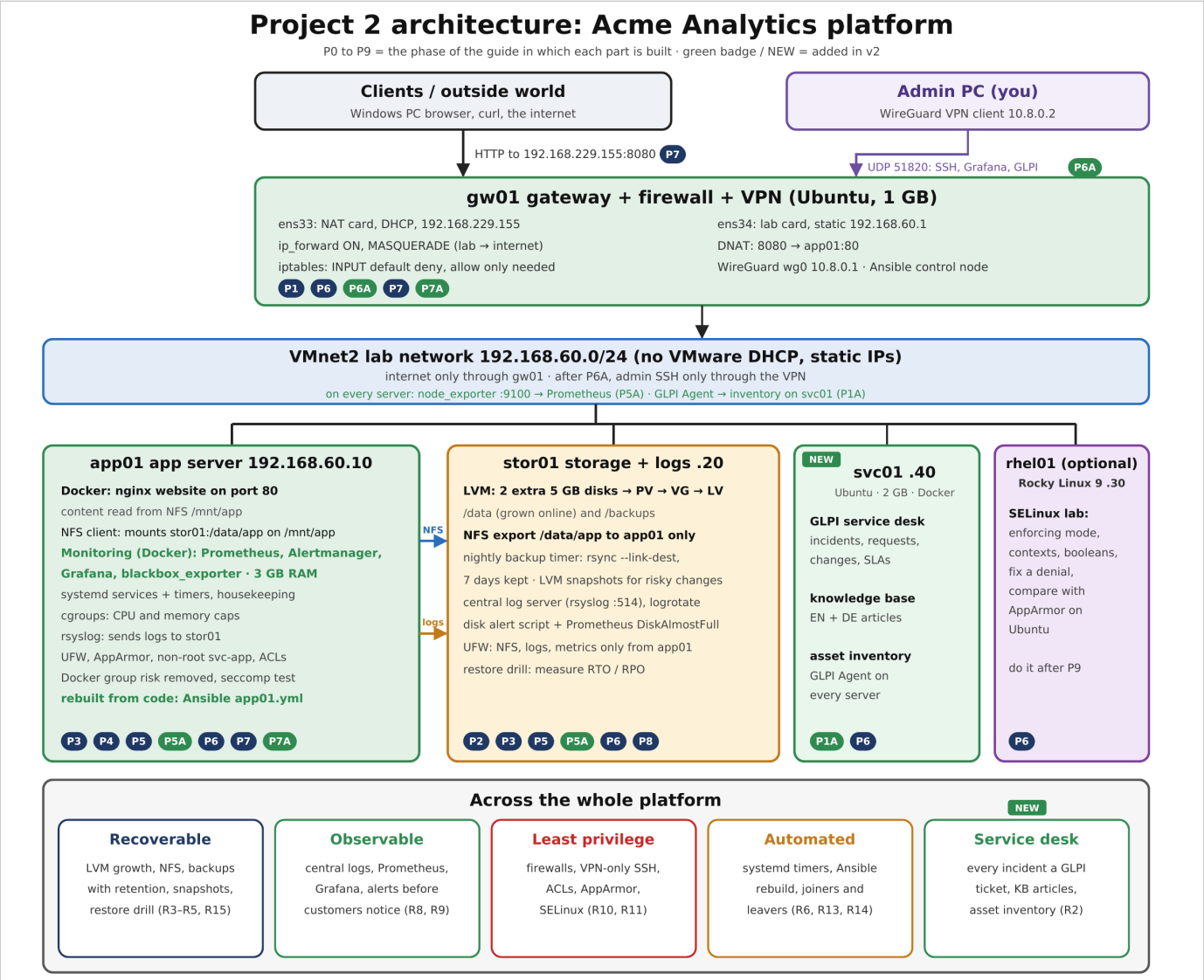
drive letter such as D: (≈ mounting). Extend Volume in Disk Management grows a volume live, like `lvextend -r`. Linux uses one folder tree under `/`, while Windows uses drive letters, but the layers are the same: disk → pool → volume → file system → mount point. On a normal PC like mine the disk is “Basic”: only partitions with drive letters, no pool – storage pools are used on servers with several disks.

For the service desk, Docker takes **images** (the installers for GLPI and MySQL) and runs them as **containers**. Their data is saved in **mounted folders** on svc01, for example `/opt/helpdesk/storage/mysql`, which appears inside MySQL as `/var/lib/mysql`. So tickets, users and inventory stay safe on `/` even if a container is deleted or upgraded, and that is why I grew the disk before installing GLPI.

6. The network story – from the first picture to today

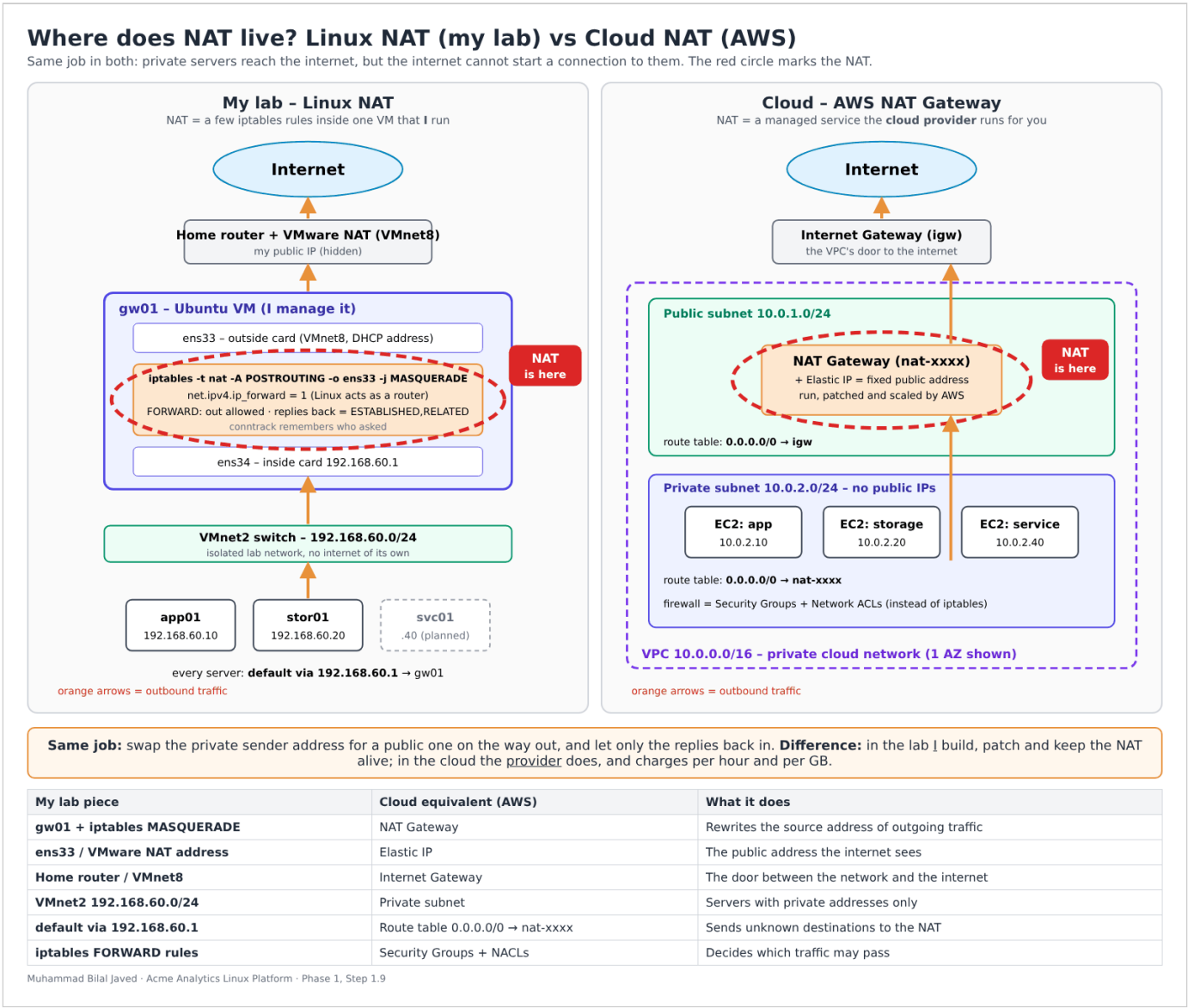
💡 **Read this section alone and you have the whole network in your head – no need to go back to Day 1.**

Picture 1 – the plan. Acme gets four servers on one private lab network, and one gatekeeper (gw01) between that network and the internet.



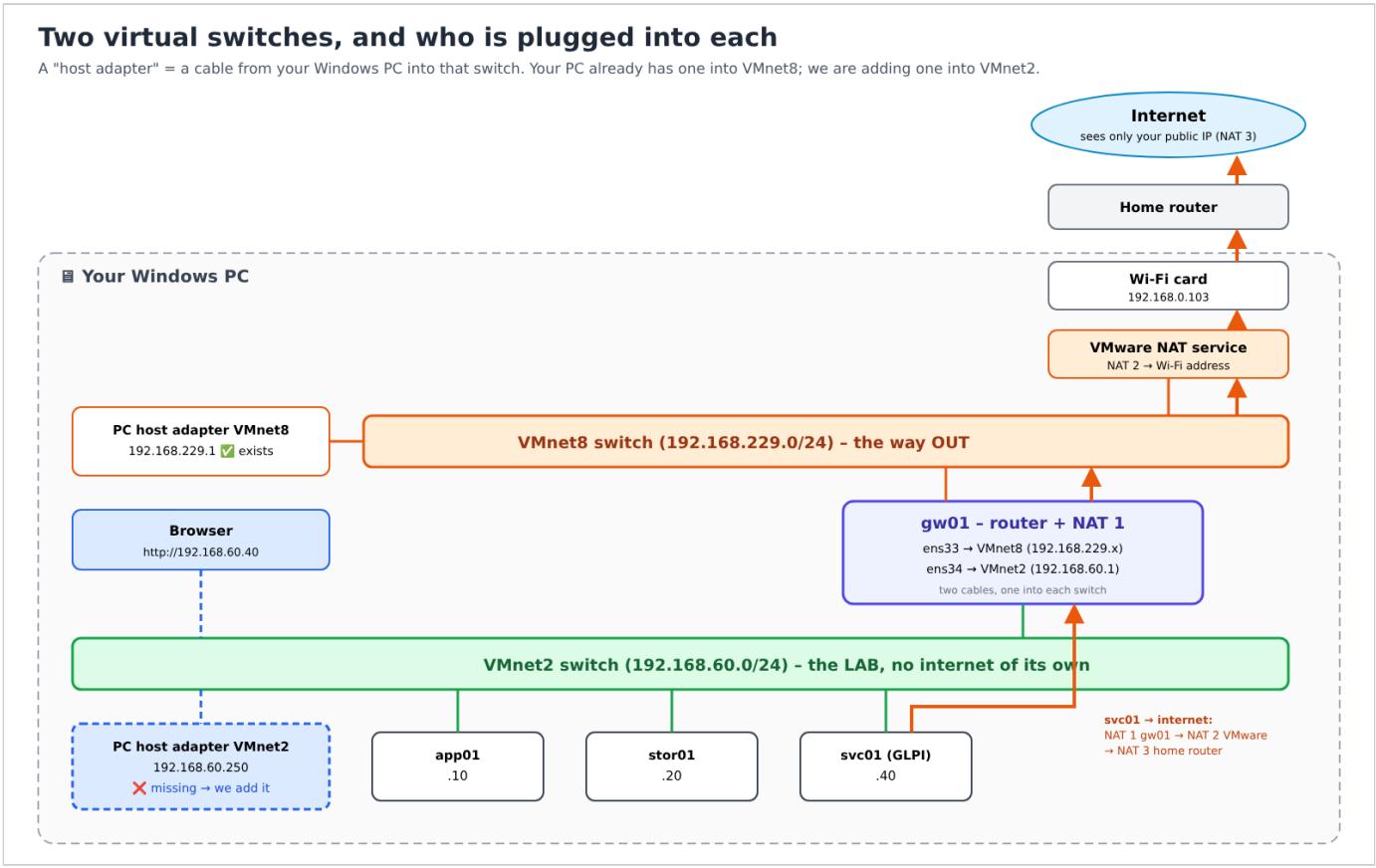
Project architecture

Picture 2 – the gatekeeper (Phase 1). In the beginning every VM had its own NAT card straight to the internet. That is easy but unsafe: nobody controls the traffic. So the servers’ outside cards were removed. Only **gw01** keeps two cards: **ens33** towards VMware’s **VMnet8** (the way out) and **ens34** = `192.168.60.1` on **VMnet2** (the lab). With IP forwarding and an iptables MASQUERADE rule, gw01 became a router with NAT – the same job a cloud NAT gateway does.



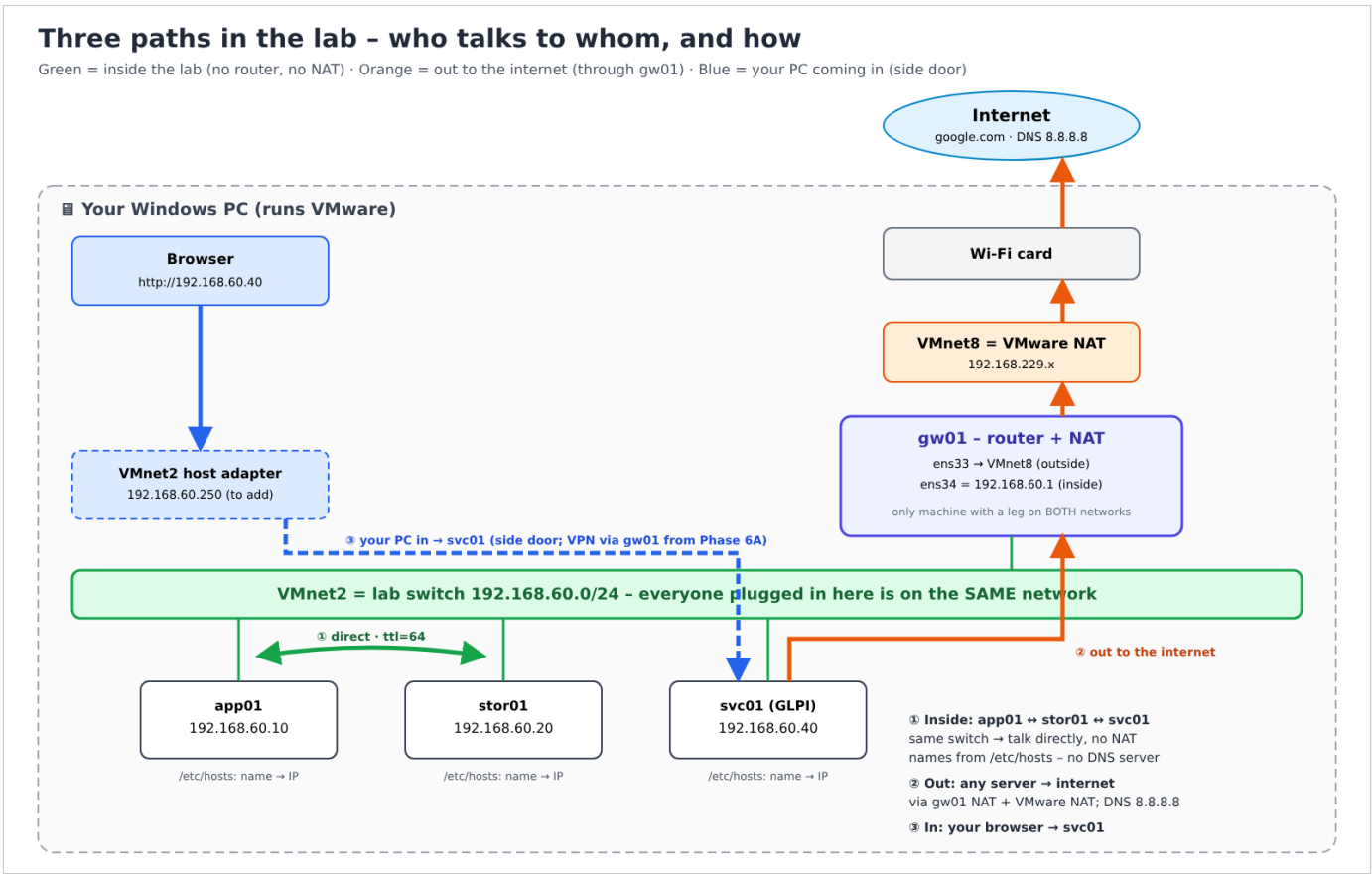
Linux NAT vs cloud NAT

Picture 3 – two virtual switches. VMware builds switches in software. **VMnet2** is the lab switch: app01, stor01, svc01 and gw01's inside card are plugged into it, so they talk to each other **directly** (no router, no NAT – `ttl=64`); `/etc/hosts` only turns their names into IPs. **VMnet8** is the way out: only gw01's outside card is plugged into it. A *host adapter* is a cable from my Windows PC into one switch – it already had one into VMnet8, but **none into VMnet2**.



Two switches and the host adapters

Picture 4 – three paths. ① **Inside:** server ↔ server over VMnet2, directly. ② **Out:** a server → gw01 (**NAT 1**, to gw01's VMnet8 address) → VMware NAT (**NAT 2**, to my Wi-Fi card's address) → home router (**NAT 3**, to my public IP) → internet. DNS (8.8.8.8) is on the internet too, so name lookups for google.com take the same path – that is why nothing worked while gw01 was off. ③ **In:** my browser → svc01. This needs my PC on the lab switch, so the **VMnet2 host adapter** is added with the free address `192.168.60.250` (not .1 – that is gw01). It is a local side door for this one PC, not a VPN; in **Phase 6A** it is replaced by a WireGuard VPN through gw01 – the way a real remote worker comes in.



Three paths in the lab

Why today: GLPI now runs on svc01 and answers `200 OK` from svc01 itself and from gw01. To use the service desk in a browser, my PC needs path ③.

How names are found (two places, in order – `hosts: files dns` in `/etc/nsswitch.conf`):

Order	Where Linux looks	Lab names (stor01)	Internet names (google.com)
1	`/etc/hosts` (local list)	✅ found here	❌ not there
2	DNS via the helper 127.0.0.53 → 8.8.8.8	❌ no DNS server knows the lab	✅ found here (and cached)

Switch, router, bridge:

Device	Like	In the lab
Switch	an extension board – devices on the same network plug in and talk directly	VMnet2 (lab board), VMnet8 (way-out board)
Router	a person between two boards passing messages across	gw01 – one card on each board
Bridge (VMware “Bridged”)	plugging a VM straight into the home network	not used – the lab stays separate

Network adapter (VMware) = `ens34` / `ens33` (Linux) = network card (NIC) – one thing, three names.

> **Key understanding (in my own words).** Every server has a network card. VMware calls it a “Network Adapter”. Inside Linux the card is called `ens34` or `ens33`.
>
> A virtual switch is like an extension board. VMnet2 is the lab board. VMnet8 is the board for the way out.
>
> app01, stor01 and svc01 are plugged into VMnet2. gw01 is also plugged into VMnet2 with its inside card. All of them have an address in the same network. So they talk to each other directly by IP. This worked from the first day.
>
> To use names, Linux looks in two places. First it checks `/etc/hosts`. Then it asks DNS. Our lab names are written in `/etc/hosts`, so they are found in the first place. Internet names like google.com are not in `/etc/hosts`. So Linux asks DNS. The helper 127.0.0.53 forwards the question to 8.8.8.8 and keeps the answer in a cache.
>
> For the internet, the servers send their traffic to gw01. gw01 has a second card on VMnet8. The address is changed three times on the way out. First gw01 changes it to its own address on VMnet8. Then VMware’s NAT changes it to my Wi-Fi card’s address. Then my home router changes it to my public IP. The DNS question to 8.8.8.8 also takes this way. So nothing on the internet works while gw01 is off.
>
> My Windows PC had a card on VMnet8 but no card on VMnet2. So my browser could not reach svc01. Now I give my PC a card on VMnet2 with the tick “Connect a host virtual adapter”. The card gets the address 192.168.60.250. My browser can now open GLPI on svc01 directly. Nothing changes for the servers.

7. Next steps

1. `sudo poweroff`, remove the NAT adapter, snapshot `24-svc01-installed`; check `ip -br a` shows only `lo` and `ens34`.
2. Step 1A.2: install Docker and start GLPI.